

Supply Chain Security

Purpose

Modern software depends on third-party code packages, container images, operating system packages, build tools, and transitive dependencies. These components help teams move quickly, but they also create risk when packages are downloaded directly from public repositories, introduced by AI coding tools, or used without review of origin, maintenance, licensing, and vulnerability posture.

The goal is not to make development harder. The goal is to give developers a clear, approved path for using packages safely: where to download them, how new packages are approved, how vulnerabilities are fixed, and when hardened or backported packages must be used.

Definitions

Software supply chain: The people, processes, tools, source code, packages, build systems, container images, infrastructure definitions, and deployment workflows used to create and operate software.

Code package: A third-party library, module, dependency, framework, plugin, container image, operating system package, build tool, or other reusable software component used by an application or build process.

Approved package source: An enterprise-recognized package repository, mirror, proxy, or hardened package provider used to download and govern code packages. Examples may include JFrog Artifactory, Nexus, Chainguard, Seal Security, or another approved platform.

Approved package: A package, version, and source that has been reviewed and allowed for use based on business need, security posture, license requirements, maintainership, and vulnerability status.

Backported fix: A security patch applied to an older package version without requiring the application to upgrade to a newer major or minor release. Backported fixes may be useful when an immediate upstream upgrade would create application compatibility risk. Red Hat Enterprise Linux (RHEL) is a great example of a vendor that backports security fixes.

Why This Matters

Developers need packages to build useful software. However, public package ecosystems can be abused through compromised maintainer accounts, malicious updates, typosquatting, dependency confusion, abandoned packages, risky install scripts, and vulnerable transitive dependencies.

AI coding assistants can make this easier to miss because they may add packages quickly and confidently. A developer may not know whether an AI-suggested package is popular, maintained, recently created, typosquatted, compromised, or pulling in risky child dependencies.

Common risks include:

- Developers download packages directly from public registries with no enterprise visibility.
- Build systems pull the newest package version automatically without review.
- Package manifests and lockfiles are missing from source control.
- AI tools introduce unnecessary or unfamiliar dependencies.

- New packages are used before AppSec, legal, or platform teams can review them.
- Critical vulnerabilities remain unresolved because application teams cannot safely upgrade.
- Direct downloads bypass enterprise package blocking, malware detection, license checks, and audit logging.
- Compromised packages execute on laptops, CI/CD runners, or build environments that contain sensitive credentials.

The right control model should reduce risk without blocking normal development. Developers should know which package source to use, how to request a new package, and how to remediate vulnerabilities when they appear.

Security Position

Application teams should use approved code packages from approved package sources. Direct package downloads from public repositories should be avoided unless an exception is documented and approved.

The organization should provide a secure development "happy path" that works for normal developer workflows. If developers use npm, PyPI, Maven, NuGet, Go modules, RubyGems, container registries, or operating system package managers, those tools should be configured to use approved repositories, mirrors, proxies, or hardened providers wherever practical.

Approved package platforms should support the following outcomes:

- Developers can download packages through a standard, documented process.
- Package inventory, versions, and usage are visible to AppSec and platform teams.
- New package requests can be reviewed before adoption.
- Known malicious, suspicious, unlicensed, or critically vulnerable packages can be blocked.
- Vulnerability remediation can be tracked and enforced.
- Hardened packages or backported fixes can be used when required.

Possible technical approaches include JFrog Artifactory, Nexus, Chainguard, Seal Security, or another approved package management and hardening platform. The specific platform matters less than the control outcome: packages should be visible, governed, reviewable, and fixable.

Required Baseline Practices

Projects should follow these baseline practices unless an exception is documented and approved.

1. Use approved package sources

Developers, build systems, CI/CD jobs, containers, and deployment workflows should download packages from approved package sources. Package managers should be configured to use the approved registry, mirror, proxy, or hardened provider for the relevant ecosystem.

2. Avoid direct public downloads

Teams should avoid downloading packages directly from public registries, GitHub release URLs, random vendor websites, or shell installer commands unless that path has been reviewed and approved. Direct downloads reduce visibility and can bypass blocking, scanning, license review, and audit logging.

3. Keep manifests and lockfiles in source control

Package manifests and lockfiles must be committed to approved source control so dependency changes are visible, reviewable, and scannable. Examples include `package.json`, `package-lock.json`, `yarn.lock`, `pnpm-lock.yaml`, `requirements.txt`, `Pipfile.lock`, `poetry.lock`, `pom.xml`, `build.gradle`, `packages.lock.json`, `go.mod`, `go.sum`, `Gemfile.lock`, and container image definitions.

4. Review dependencies introduced by AI agents

Developers should understand what packages their AI tools add to a project. They should know why each package is needed, whether an approved alternative exists, what license applies, and whether the package has risky transitive dependencies.

5. Prefer stable and necessary dependencies

Teams should avoid unnecessary packages, abandoned packages, newly published packages with little adoption, and bleeding-edge versions unless there is a clear business or security reason. Dependency cooldowns may be used to prevent automatic adoption of brand-new releases before the ecosystem has had time to detect malicious updates.

6. Request approval for new packages

If a required package is not already approved, the developer should submit a package request before using it in production software. The request should include the package name, ecosystem, version or version range, business purpose, source URL, license, maintainer information, alternatives considered, and whether it was recommended by an AI tool.

7. Audit new packages before approval

AppSec, platform, legal, or other designated reviewers should evaluate requested packages before approval. Reviews should consider known vulnerabilities, license risk, maintainer reputation, package age, release history, transitive dependencies, install scripts, popularity, signs of typosquatting or dependency confusion, and whether a safer approved alternative exists.

8. Use approved packages consistently

Once a package is approved, teams should use the approved source and approved version or version range. Approval of one package does not automatically approve similarly named packages, forks, plugins, major version changes, or new package sources.

9. Fix vulnerable packages according to severity

Vulnerable packages must be remediated according to standard vulnerability management expectations. Critical vulnerabilities should be fixed, mitigated, removed, replaced, or covered by an approved risk exception before release.

10. Use backported fixes when required

If a package has a critical vulnerability and the application cannot safely upgrade to a non-vulnerable upstream version in the required timeframe, the team must use an approved hardened or backported fix when one is available. Seal Security's backported fixes may be required for these cases when approved by AppSec and platform teams.

11. Reduce development and build blast radius

Projects that use sensitive data, privileged credentials, internal APIs, or production-adjacent systems should use isolated development and build environments where practical, such as GitHub Codespaces, stripped-down virtual machines, dedicated CI/CD runners, or sandboxed build containers. This lowers the impact if a malicious package, install script, or build tool executes.

12. Use minimal and trusted container images

Containerized applications should use approved base images and include only the packages needed for the application to run. Hardened or minimal images, such as those from Chainguard or another approved provider, should be considered for higher-risk services.

13. Protect secrets from package execution

Package install and build steps should not run with broad access to developer secrets, production credentials, personal files, or administrative tokens. If a dependency install process is compromised, response should not require broad credential rotation across unrelated systems.

14. Document exceptions

Exceptions should identify the package, version, source, business justification, risk, compensating controls, expiration date, and owner. Exceptions should not become permanent approval by default.

How Developers Download Packages

Developers should use normal package manager commands, but those tools should resolve packages through approved package sources.

Examples include:

- npm, Yarn, or pnpm configured to use an approved JavaScript package registry or proxy.
- pip, Poetry, or Pipenv configured to use an approved Python package index or proxy.
- Maven or Gradle configured to use an approved Maven repository or proxy.
- NuGet configured to use an approved NuGet feed.
- Go modules configured to use approved proxy and checksum settings where practical.
- Docker and container tooling configured to use approved image registries and base images.
- Operating system package managers configured to use approved mirrors or hardened package sources where practical.

Developers should not work around the approved source because a public package command is faster or because an AI assistant suggested it. If the package is missing, the developer should request the package.

How Packages Get Approved

The package approval process should be lightweight enough that developers use it, but strong enough to prevent high-risk packages from entering enterprise software unnoticed.

Recommended approval workflow:

1. Developer identifies a needed package and checks whether it is already available from an approved source.

2. If the package is not available, the developer submits a package request.
3. The request includes the package name, ecosystem, version, purpose, project, owner, source URL, license, alternatives considered, and urgency.
4. Reviewers evaluate security, license, maintainership, transitive dependency, and operational risk.
5. Reviewers decide whether to approve, approve with conditions, require a safer alternative, require a hardened or backported version, deny, or grant a time-bound exception.
6. Approved packages are made available through the approved package source.
7. The approval decision is recorded so future teams can understand what was approved and under what conditions.

Approval should be version-aware. A previously approved package may need renewed review when a major version changes, maintainership changes, a package is deprecated, a license changes, or new vulnerability information appears.

How Developers Fix Vulnerabilities

When a package vulnerability is identified, the application team should choose the safest remediation that fits the application and the vulnerability severity.

Preferred remediation options include:

- Upgrade to a non-vulnerable version from the approved package source.
- Replace the package with an approved safer alternative.
- Remove the package if it is unnecessary.
- Apply configuration or code changes that eliminate the vulnerable usage.
- Use an approved hardened or backported package when an immediate upstream upgrade is not practical.
- Create a documented risk exception only when remediation or mitigation cannot be completed in the required timeframe.

Critical vulnerabilities require special handling. If a critical vulnerability exists in a package and the team cannot safely upgrade or remove it quickly enough, the team must use an approved backported fix when available. Seal Security's backported fixes should be used for these cases when that is the approved remediation path.

Vulnerability fixes should be reflected in source control through updated manifests and lockfiles. Teams should not silently change package versions only on developer machines, CI/CD runners, or production hosts.

Relevant Practices from Citizen Developer and Vibe Coding Guidance

The following practices from the citizen developer and vibe coding guidance also apply directly to supply chain security:

- Code should be stored in approved Git repositories so dependency manifests, lockfiles, and package changes are visible.
- Repositories should be onboarded to available Wiz SAST, SCA, IaC, and secrets scanning where supported.
- Developers should review libraries and packages introduced by AI agents before accepting them.

- Package managers, package names, versions, maintainers, licenses, and transitive dependencies should be visible through source control and dependency scanning.
- Higher-risk projects should use sandboxed development environments such as GitHub Codespaces or stripped-down virtual machines.
- Applications should be containerized where appropriate, with minimal base images and only necessary runtime packages.
- Enterprise package management, repository mirroring, or artifact management platforms such as Nexus or JFrog Artifactory should be used to improve visibility and control.
- Hardened package providers such as Seal Security or Chainguard should be considered where appropriate.
- Dependency cooldowns should be considered to avoid automatically pulling brand-new package versions before malicious releases can be detected.
- Stable, non-bleeding-edge dependency versions should be preferred unless a project has a clear reason to adopt a newer version.
- The SOC team, Wiz Code, Wiz Mika, source control history, package lockfiles, CI/CD logs, and endpoint telemetry may be needed to investigate compromised packages or campaigns.
- Credential rotation may be required when a compromised package runs in an environment that had access to secrets.

Escalation Criteria

Additional review or escalation may be needed when a project:

- Depends on newly published, unusual, unreviewed, or AI-selected third-party packages.
- Uses packages with critical or actively exploited vulnerabilities.
- Requires direct downloads from public package sources or vendor websites.
- Uses package install scripts that execute code during installation.
- Pulls packages during CI/CD builds without using an approved package source.
- Runs package installation on developer laptops with access to sensitive credentials.
- Uses privileged service accounts, production credentials, or administrative tools during build or install steps.
- Uses container images or operating system packages from unapproved sources.
- Cannot produce dependency manifests, lockfiles, or a reliable software bill of materials.
- Cannot remediate a critical vulnerability within the required timeframe.

Escalation should focus on reducing risk and helping the team reach an approved path, not blocking useful development without an alternative.

Practical Package Request Checklist

Use this checklist when requesting a new package:

- Package name:
- Package ecosystem:
- Requested version or version range:
- Project or application:
- Business owner:
- Technical owner:

- Business purpose:
- Source URL:
- Approved package source requested:
- License:
- Maintainer or publisher:
- Package age and release history:
- Alternatives considered:
- Reason existing approved packages are not sufficient:
- Was the package suggested by an AI tool:
- Direct dependencies:
- Notable transitive dependencies:
- Known vulnerabilities:
- Critical vulnerabilities present:
- Hardened or backported version available:
- Install scripts or build-time execution:
- Container image or base image, if applicable:
- Sensitive data, credentials, or production systems involved:
- Requested approval duration:
- Reviewer decision:
- Conditions or compensating controls:
- Next review date:

Summary

Supply chain security is about making safe package use the easiest path. Developers should know where packages come from, how new packages are approved, and how vulnerabilities are fixed.

Approved package sources, package review, source-controlled lockfiles, automated scanning, sandboxed development, minimal containers, and backported fixes all serve the same purpose: reduce the chance that an unsafe package reaches production, and reduce the blast radius if a package is compromised anyway.